Smart Agricultural Production Optimization Engine(OptiCrop) (Using Machine Learning)

Project Description:

The Smart Agricultural Production Optimization Engine (OptiCrop) is an AI-powered agricultural decision support system designed to optimize crop production using machine learning techniques. The application analyzes important environmental and soil parameters such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH level, rainfall, and crop type to provide intelligent crop recommendations.The system employs machine learning algorithms to analyze agricultural datasets and identify patterns between environmental conditions and crop productivity. By leveraging data-driven insights, OptiCrop helps farmers maximize crop yield, improve resource utilization, and promote sustainable farming practices.

The platform also assists agricultural researchers and policymakers in understanding crop-environment interactions and developing evidence-based agricultural strategies.

Problem Statement

Traditional farming practices often rely on farmers' experience and manual analysis, which may lead to inefficient resource utilization and lower crop yields. Factors such as soil nutrients, weather conditions, and rainfall significantly influence agricultural production, making it difficult to determine the most suitable crop.

There is a need for an intelligent system that can analyze environmental and soil conditions and provide accurate crop recommendations. The proposed system addresses this challenge by applying machine learning techniques to optimize agricultural production and improve decision-making.

Scenarios:

Scenario 1: Smart Crop Recommendation for Farmers

A farmer enters soil and environmental information, including Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH level, and rainfall. The system processes the data and recommends the most suitable crop for maximizing productivity and farming efficiency.

Scenario 2: Crop Suitability and Environmental Assessment

A farmer or agricultural expert evaluates whether current soil and climate conditions are suitable for cultivating a specific crop. The system analyzes the environmental parameters and provides recommendations regarding crop compatibility and productivity.

Scenario 3: Agricultural Research and Policy Planning

Researchers and policymakers use the platform to study crop-environment relationships and identify agricultural trends. The system supports data-driven decision-making for sustainable farming practices and efficient resource management.

Objectives

- To analyze agricultural datasets containing soil and environmental information.
- To preprocess and clean agricultural data for machine learning applications.
- To develop predictive models that recommend suitable crops.
- To improve agricultural productivity and resource utilization.
- To assist farmers in making informed crop selection decisions.
- To support agricultural researchers and policymakers with data-driven insights.
- To evaluate model performance using standard machine learning metrics.
- To deploy the model through a user-friendly web application.

Scope

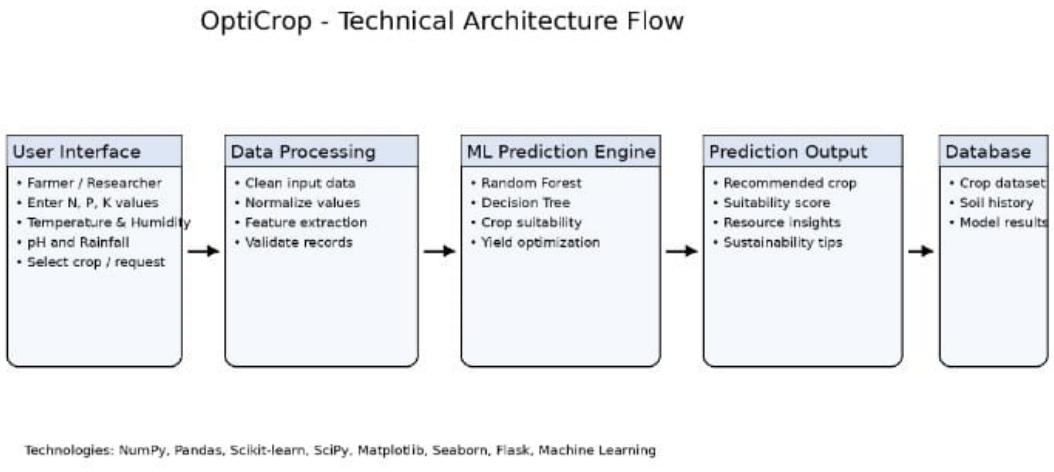
The scope of this project is limited to to crop recommendation and agricultural production optimization based on environmental and soil conditions. The system demonstrates the complete machine learning workflow, including:

- Data collection and preprocessing.
- Feature engineering.
- Model training and evaluation.
- Prediction generation.
- Web application deployment.

The project is intended for educational and research purposes and can be extended for real-world agricultural applications.

Technical Architecture:

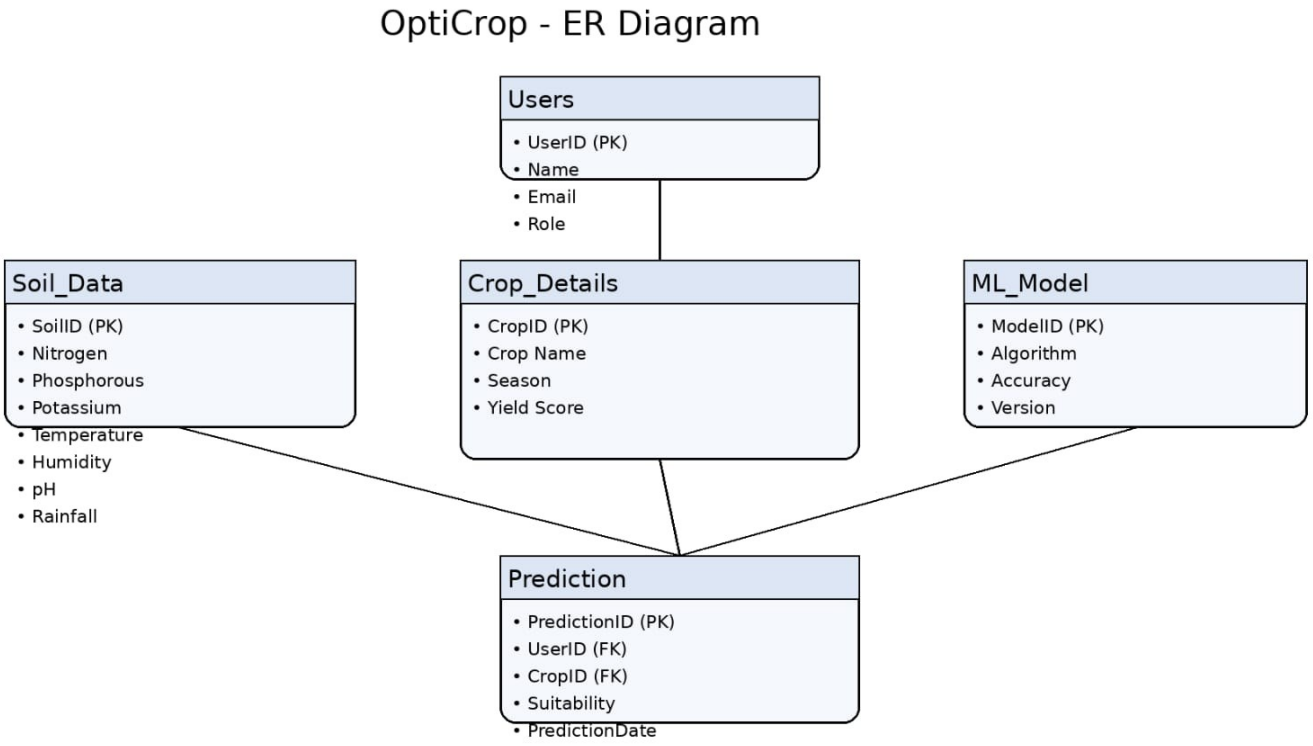
Architecture Flow



ER Diagram

Description:

The Smart Agricultural Production Optimization Engine follows a modular architecture consisting of multiple interconnected components.



Pre-requisites

- Python Programming Proficiency (Python 3.x)
- Machine Learning Fundamentals — scikit-learn, XGBoost
- Data Handling Libraries — Pandas, NumPy
- Data Visualization — Matplotlib / Seaborn (for EDA), SciPy
- Web Framework Knowledge — Flask, for deployment of the prediction interface
- Version Control with Git
- Development Tools— Jupyter Notebook, Visual Studio Code
- Database — CSV Dataset

Project Workflow:

Milestone 1: Data Collection and Understanding :

- Activity 1.1: Collect agricultural datasets containing soil nutrients and environmental parameters.
- Activity 1.2: Analyze dataset features.
- Activity 1.3: Understand relationships between crops and environmental conditions.

Milestone 2: Data Preprocessing and Feature Engineering

- Activity 2.1: Handle missing values.
- Activity 2.2: Remove duplicates.
- Activity 2.3: Normalize data.
- Activity 2.4: Encode categorical variables.
- Activity 2.5: Select important features.
- Activity 2.6: Split the dataset into training and testing sets.
- Activity 2.7: Apply StandardScaler to numerical features where appropriate, to normalize feature ranges for scale-sensitive algorithms.

Milestone 3: Model Building

- Activity 3.1: Train machine learning models.
- Activity 3.2: Compare model performance.
- Activity 3.3: Select the best-performing model.
- Possible algorithms: Random Forest, Decision Tree, Logistic Regression, K-Nearest Neighbors (KNN), Support Vector Machine (SVM)

Milestone 4: Model Evaluation

- Evaluate models using: Accuracy, Precision, Recall, F1-Score, Confusion Matrix.

Milestone 5: Web Application Development

- Activity 5.1: Build the Flask-based web application.
- Activity 5.2: Integrate the trained model.
- Activity 5.3: Deploy the prediction interface.
- Activity 5.4: Deploy the application.

The application has been successfully deployed and is live at the following URL:

https://opticrop-2lj9.onrender.com

Methodology

Dataset Description

Attribute	Details
Domain	Agriculture
Dataset	Croprecommendation_dataset.csv
Framework	Flask
Type	Crop Recommendation

Data Preprocessing Steps

- Load agricultural dataset using Pandas.
- Handle missing and duplicate values.
- Normalize environmental parameters.
- Perform exploratory data analysis.
- Split data into training and testing datasets.
- Train machine learning models.
- Evaluate model performance.
- Save the trained model.

Application Used

- Python
- Jupyter Notebook
- Visual Studio Code
- Flask
- GitHub

Evaluation Metrics

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix
- Cross Validation Score

Web Application

The web application allows users to input:

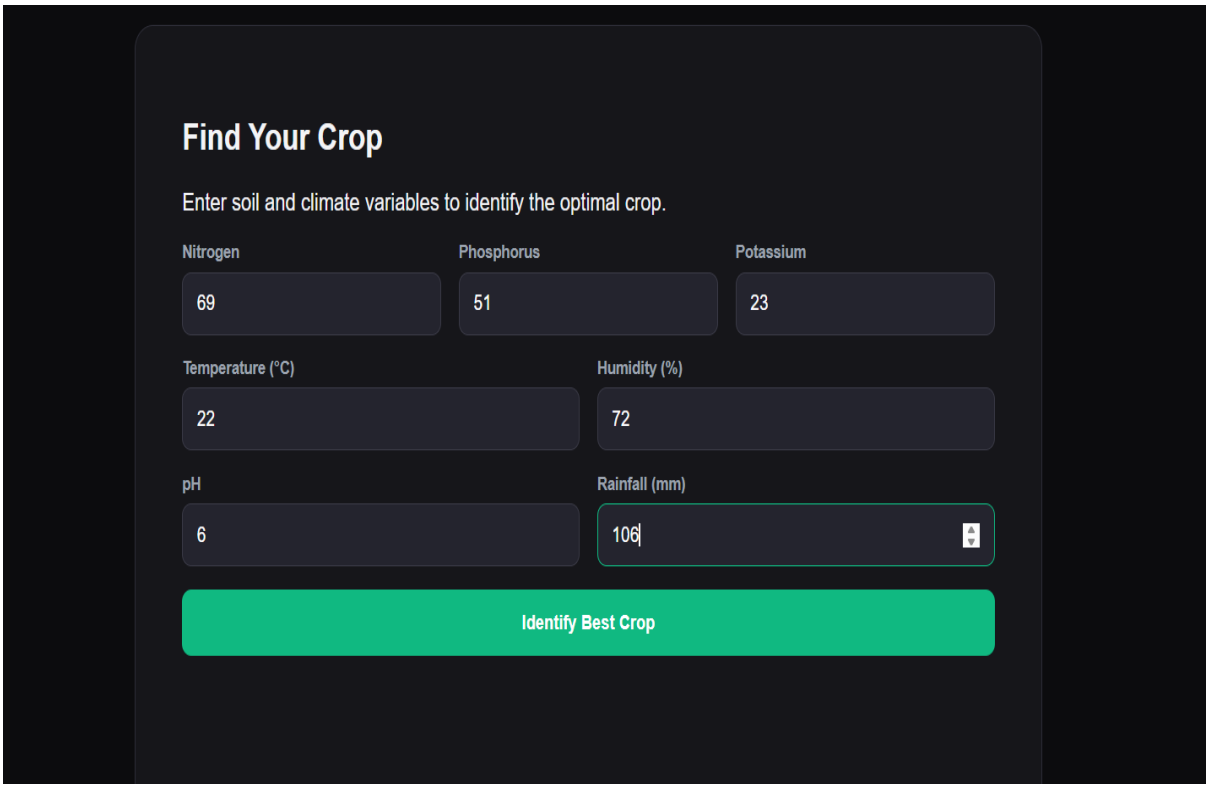
- Nitrogen (N)
- Phosphorous (P)
- Potassium (K)
- Temperature
- Humidity
- pH Value
- Rainfall

After processing the data, the application recommends the most suitable crop and displays the result instantly.

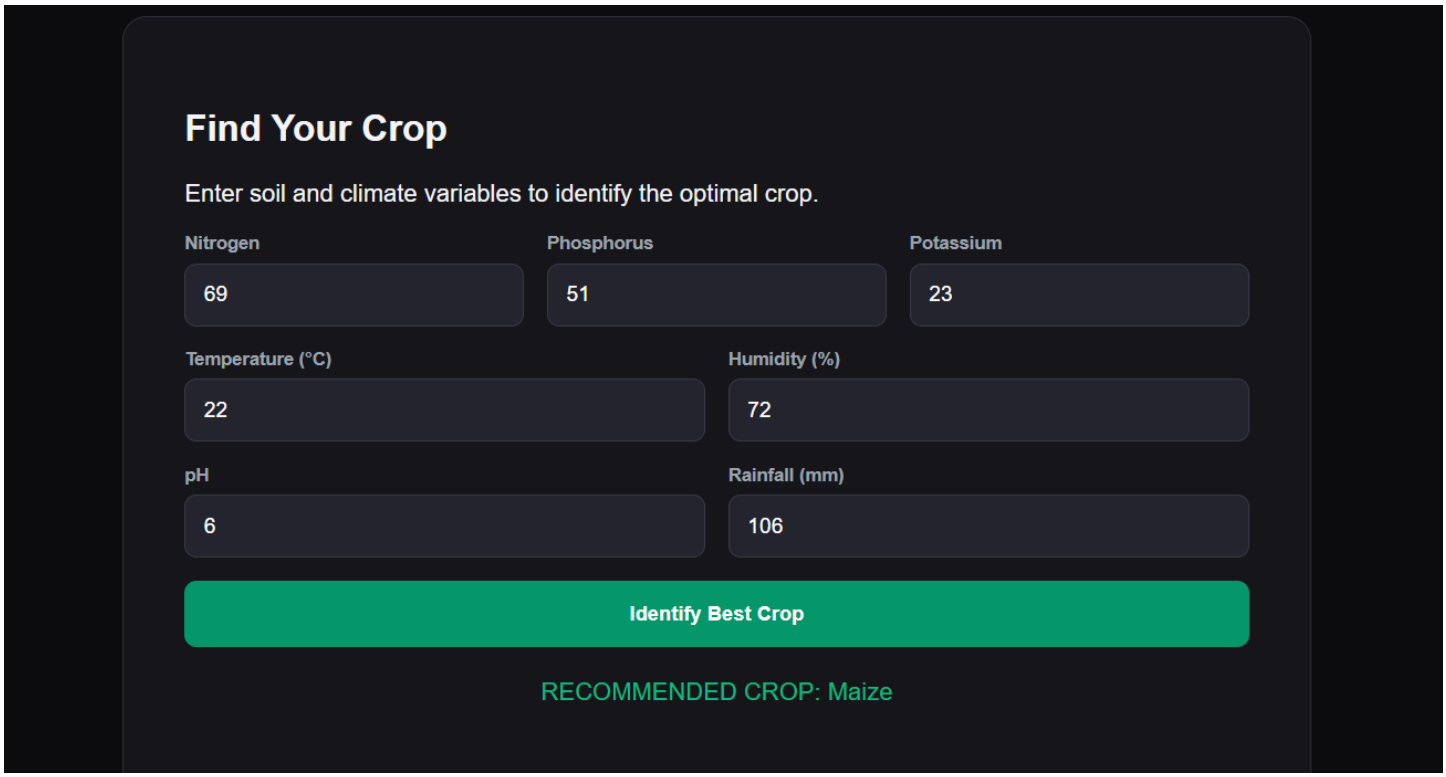
The application has been successfully deployed and is publicly accessible at:

https://opticrop-2lj9.onrender.com

Application Form



Prediction of Result



Frontend Implementation

- HTML
- CSS
- JavaScript

HTML(structure)

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>OptiCrop</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">

    <style>
        .hero {
            background: url("{{ url_for('static', filename='farm.png') }}") center/cover no-repeat;
        }
    </style>

</head>

<body>

<!-- NAVBAR -->
<nav class="navbar">
    <div class="logo">OptiCrop</div>
    <ul>
        <li><a href="#home" class="active">Home</a></li>
        <li><a href="#about">About</a></li>
        <li><a href="#predict">FindCrop</a></li>
    </ul>
</nav>

<!-- HOME -->
<section id="home" class="hero">
    <div class="overlay">
        <h1 class="hero-title">OptiCrop Agricultural Optimisation</h1>
        <p class="hero-text">
            The main purpose of OptiCrop is to develop a system that uses
            data-driven insights to optimize agricultural production.
        </p>
    </div>
</section>

<!-- ABOUT -->
<section id="about" class="about">
    <div class="about-container">

        <!-- LEFT TEXT -->
        <div class="about-text">
            <h2>Smart Agricultural Optimization Using Machine Learning</h2>

            <p>
                OptiCrop is an intelligent crop recommendation system designed
                to assist farmers in selecting the most suitable crops based on
                soil nutrients, environmental conditions, and climate data.
            </p>

            <p>
                By analyzing parameters such as Nitrogen (N), Phosphorus (P),
                Potassium (K), temperature, humidity, pH, and rainfall,
                OptiCrop uses machine learning models to provide accurate
                and reliable crop suggestions.
            </p>
```

```
<!-- FIND YOUR CROP -->
<section id="predict" class="predict">
    <div class="card">
        <h2 class="title">Find Your Crop</h2>
        <p class="subtitle">
            Enter soil and climate variables to identify the optimal crop.
        </p>

        <form id="predictionForm">

            <!-- Soil Nutrients -->
            <div class="row">
                <div class="col">
                    <label for="N">Nitrogen</label>
                    <input type="number" id="N" name="N" required>
                </div>
                <div class="col">
                    <label for="P">Phosphorus</label>
                    <input type="number" id="P" name="P" required>
                </div>
                <div class="col">
                    <label for="K">Potassium</label>
                    <input type="number" id="K" name="K" required>
                </div>
            </div>

            <!-- Climate Conditions -->
            <div class="row">
                <div class="col">
                    <label for="temperature">Temperature (°C)</label>
                    <input type="number" id="temperature" name="temperature" step="any" required>
                </div>
                <div class="col">
                    <label for="humidity">Humidity (%)</label>
                    <input type="number" id="humidity" name="humidity" step="any" required>
                </div>
            </div>

            <!-- Soil Properties -->
            <div class="row">
                <div class="col">
                    <label for="ph">pH</label>
                    <input type="number" id="ph" name="ph" step="any" required>
                </div>
                <div class="col">
                    <label for="rainfall">Rainfall (mm)</label>
                    <input type="number" id="rainfall" name="rainfall" step="any" required>
                </div>
            </div>

            <button type="submit">Identify Best Crop</button>
        </form>

        <div id="result"></div>
    </div>
</section>

<script src="{{ url_for('static', filename='script.js') }}"></script>

</body>
```

CSS

```css
body {
    background-color: #0c0c0e;
    color: #f3f4f6;
    font-family: Arial, sans-serif;
    margin: 0;
}

/* SMOOTH NAVIGATION */
html {
    scroll-behavior: smooth;
}

/* NAVBAR */
.navbar {
    position: fixed;
    top: 0;
    width: 100%;
    background: #16161a;
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 16px 20px;
    border-bottom: 1px solid #2e2e38;
    z-index: 1000;
    box-sizing: border-box;
}

/* BIG TITLE FIX */
.logo {
    font-size: 40px;
    font-weight: bold;
    color: #2ecc71;
    letter-spacing: 1px;
}

/* NAV LINKS */
.navbar ul {
```

```css
.about-container {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 40px;
    max-width: 1100px;
    margin: auto;
}

/* LEFT TEXT */
.about-text {
    flex: 1;
}

.about-text h2 {
    font-size: 28px;
    margin-bottom: 20px;
}

.about-text p {
    font-size: 16px;
    line-height: 1.6;
    margin-bottom: 15px;
}

/* RIGHT IMAGE */
.about-image {
    flex: 1;
}

.about-image img {
    width: 100%;
    border-radius: 12px;
    object-fit: cover;
}

/* FORM SECTION */
.predict {
    display: flex;
    justify-content: center;
}

/* CARD */
.card {
    background-color: #16161a;
    border: 1px solid #2e2e38;
    padding: 40px;
    border-radius: 16px;
    max-width: 700px;
    width: 100%;
}

/* FORM FIX (NO OVERFLOW) */

.row {
    display: flex;
    gap: 15px;
    margin-bottom: 15px;
    flex-wrap: wrap; /* KEY FIX */
}
```

JavaScript


```
document.getElementById('predictionForm').addEventListener('submit', async (e) => {
    e.preventDefault();

    // Package form parameters
    const data = {
        N: parseFloat(document.getElementById('N').value),
        P: parseFloat(document.getElementById('P').value),
        K: parseFloat(document.getElementById('K').value),
        temperature: parseFloat(document.getElementById('temperature').value),
        humidity: parseFloat(document.getElementById('humidity').value),
        ph: parseFloat(document.getElementById('ph').value),
        rainfall: parseFloat(document.getElementById('rainfall').value)
    };

    const resultBox = document.getElementById('result');
    resultBox.className = '';
    resultBox.innerText = 'Analyzing soil parameters...';

    try {
        const response = await fetch('/predict', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(data)
        });

        const res = await response.json();

        if (response.ok && res.success) {
            resultBox.className = 'success';
            resultBox.innerText = `RECOMMENDED CROP: ${res.crop}`;
        } else {
            resultBox.className = 'error';
            resultBox.innerText = `Prediction Error: ${res.error || 'Server rejection'}`;
        }
    } catch (err) {
        resultBox.className = 'error';
        resultBox.innerText = `Connection failed: ${err.message}`;
    }


});



const links = document.querySelectorAll(".navbar a");
const sections = document.querySelectorAll("section");

links.forEach(link => {
    link.addEventListener("click", function(e) {
        e.preventDefault();

        const target = this.getAttribute("href").substring(1);

        sections.forEach(sec => sec.classList.remove("active"));
        document.getElementById(target).classList.add("active");
    });
});
```

Backend Implementation Flask(app.py)

```python
import os
import joblib
import numpy as np
from flask import Flask, request, jsonify, render_template

app = Flask(__name__)

# Safely load the saved model and scaler

MODEL_PATH = 'models/crop_model.joblib'
SCALER_PATH = 'models/scaler.joblib'

if os.path.exists(MODEL_PATH) and os.path.exists(SCALER_PATH):
    model = joblib.load(MODEL_PATH)
    scaler = joblib.load(SCALER_PATH)
    print("Crop model and scaler loaded successfully!")
else:
    model = None
    scaler = None
    print("WARNING: Model or Scaler file not found! Run train.py first.")

@app.route('/')
def home():
    """Renders the single page application interface."""
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    """Make a prediction based on the input features."""
    if model is None or scaler is None:
        return jsonify({'error': 'Machine learning model files not loaded.'}), 500


    try:
        data = request.get_json()

        # Required fields
        required_keys = ['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']
        for key in required_keys:
            if key not in data:
                return jsonify({'error': f'Missing parameter: {key}'}), 400

    # Build feature array
        raw_features = np.array([[
            float(data['N']),
            float(data['P']),
            float(data['K']),
            float(data['temperature']),
            float(data['humidity']),
            float(data['ph']),
            float(data['rainfall'])
        ]])

        # Simple parameter boundaries validation
        n, p, k, temp, hum, ph, rain = raw_features[0]
        if not (0 <= n <= 250): return jsonify({'error': 'N content must be between 0 and 250 mg/kg.'}), 400
        if not (0 <= p <= 250): return jsonify({'error': 'P content must be between 0 and 250 mg/kg.'}), 400
        if not (0 <= k <= 300): return jsonify({'error': 'K content must be between 0 and 300 mg/kg.'}), 400
        if not (-10 <= temp <= 60): return jsonify({'error': 'Temperature must be between -10°C and 60°C.'}), 400
        if not (0 <= hum <= 100): return jsonify({'error': 'Humidity must be between 0% and 100%.'}), 400
        if not (0 <= ph <= 14): return jsonify({'error': 'Soil pH must be between 0.0 and 14.0.'}), 400
        if not (0 <= rain <= 1000): return jsonify({'error': 'Rainfall must be between 0mm and 1000mm.'}), 400


        # Scale input
        scaled_features = scaler.transform(raw_features)

        # Predict
        prediction = model.predict(scaled_features)[0]

        return jsonify({
            'success': True,
            'crop': str(prediction).capitalize()
        })

    except ValueError:
        return jsonify({'error': 'Inputs must be valid numbers'}), 400
    except Exception as e:
        return jsonify({'error': f'Server Error: {str(e)}'}), 500


if __name__ == '__main__':
```

Conclusion and Future Scope

Conclusion

The Smart Agricultural Production Optimization Engine successfully demonstrates the application of machine learning in agriculture. By analyzing soil nutrients and environmental conditions, the system provides intelligent crop recommendations that improve productivity and optimize resource utilization.

The project enables farmers to make informed decisions, enhances agricultural efficiency, and supports sustainable farming practices. It also provides valuable insights for researchers and policymakers in understanding crop-environment relationships.

Overall, OptiCrop proves that machine learning can significantly improve agricultural production and contribute to modern farming solutions..

Future Scope

- Integrate real-time weather APIs.
- Deploy the application on cloud platforms such as AWS or Azure.
- Develop a mobile application.
- Incorporate deep learning techniques.
- Add multilingual support for farmers.
- Include fertilizer recommendation systems.
- Integrate IoT sensors for real-time monitoring.
- Provide market price prediction features.
- Expand the dataset for improved accuracy.
- Support precision agriculture and smart farming initiatives.